Enterprise AI Cybersecurity Copilot

Interview Questions and Deep Answers

Batch 1 of 10: Project Overview, Business Context, Scope, and High-Level Architecture

Prepared for interview practice around a production-grade Balbix-like AI copilot system

> *Focus of this batch: explaining the project clearly, defending the high-level design, and showing that the system is secure, enterprise-ready, and not just a generic LLM wrapper.*

Batch 1 Coverage

- Project explanation and business value
- Primary users and real enterprise use cases
- v1 scope and what is intentionally out of scope
- High-level architecture and request flow
- RequestContext, RBAC, and tenant isolation basics
- Tools vs RAG vs memory vs LLM responsibilities
- Multi-intent handling and bounded agentic tool loops
- Core production design principles

1. Explain your project in simple terms.

Answer:

I built an enterprise AI chatbot / AI copilot for a cybersecurity platform similar to Balbix. The goal was to let enterprise security users ask natural-language questions about cyber risk, assets, vulnerabilities, exposure, remediation, reports, and uploaded documents.

Instead of manually navigating dashboards, filters, and reports, users can ask questions such as:

```
Why did our cyber risk increase this week?
Show top risky internet-facing assets.
Generate a CISO-level risk summary.
Which vulnerabilities violate our SLA policy?
```

The system combines three major capabilities:

```
1. Tool calling over live structured product data
2. RAG over PDFs, policies, reports, and product documents
3. Secure AI orchestration with RBAC, tenant isolation, audit logs, and guardrails
```

The key point is that this is not a simple chatbot. It is a secure enterprise AI copilot that respects tenant boundaries, user permissions, data freshness, and cybersecurity safety requirements.

2. Why was this chatbot needed?

Answer:

Cybersecurity platforms contain complex data: assets, vulnerabilities, exposure scores, business units, risk trends, remediation status, and reports. Security analysts and CISOs need quick answers, but manually finding those answers across dashboards can take time.

For example, a CISO may ask:

```
What are the top business risks this month?
```

A security analyst may ask:

```
Which critical vulnerabilities should I fix first?
```

Without the chatbot, users move across dashboards, filters, reports, and exports. The chatbot reduces this friction by giving a natural-language interface over the platform.

The business value is:

```
Faster risk investigation
Better vulnerability prioritization
Easier executive reporting
Higher product adoption
Reduced manual analysis effort
Improved customer experience
```

3. Why is this not just a generic ChatGPT wrapper?

Answer:

A generic ChatGPT wrapper cannot safely answer enterprise cybersecurity questions because it does not understand tenant boundaries, RBAC, data freshness, tool permissions, or sensitive customer context.

This system needs to handle:

```
Tenant isolation
RBAC
Group-based asset permissions
Live structured data
PDF/document retrieval
Prompt injection defense
DLP checks
Audit logging
Tool approval workflows
Data freshness
Citation validation
```

A generic chatbot may hallucinate or expose unauthorized information. In this design, the LLM does not directly access databases. It only requests approved tools, and every tool call is checked by the policy engine.

The responsibility split is:

```
LLM = reasoning and explanation
Tools = trusted live facts
RAG = trusted document evidence
Policy engine = permission control
Validator = safety and correctness check
```

That separation makes the system production-grade.

4. Who are the primary users of this chatbot?

Answer:

The users are enterprise users of the cybersecurity platform. The main personas are:

```
CISO / CSO
Security analyst
Vulnerability management team
Tenant admin
Super admin
Developer / engineering user
Business risk owner
Customer success / support user
```

Different personas ask different types of questions. A CISO may ask for business-level risk summaries:

> What should I tell the board this month?

An analyst may ask operational questions:

> Which assets are most exposed?

An admin may ask:

> Which teams have unresolved critical vulnerabilities?

The chatbot must support these users while only showing data they are allowed to access.

5. What were the main use cases?

Answer:

The main use cases were:

1. Cyber risk explanation
2. CRQ metric explanation
3. CTEM / exposure analysis
4. Asset risk investigation
5. Vulnerability prioritization
6. Historical risk trend analysis
7. Executive report generation
8. PDF/document Q&A
9. Policy-aware vulnerability analysis
10. Jira ticket creation with approval

Example:

> Using our vulnerability SLA policy, show which current vulnerabilities are violating SLA.

This requires both RAG and tools. RAG retrieves the SLA policy. Tools fetch current vulnerabilities. Backend logic compares vulnerability age against SLA. The LLM then explains the result clearly with citations.

6. What was the scope of v1?

Answer:

For v1, I would keep the scope focused but production-ready.

v1 includes:

> Web-app embedded chatbot
> Authenticated users only
> Tenant isolation
> RBAC-aware answers
> Tool calling over Balbix data
> RAG over uploaded PDFs and product docs
> Conversation history
> Long-term user memory for preferences

```
Report generation
Jira ticket draft creation with approval
Audit logs
Observability and evaluation
Prompt injection and DLP guardrails
```

v1 intentionally excludes:

```
Slack / Teams integration
SOAR automation
Autonomous remediation
Fine-tuning on customer data
External SIEM / EDR integrations
Fully autonomous action execution
```

The reason is that enterprise cybersecurity systems should prioritize safety, correctness, and secure data boundaries before adding too many integrations.

7. What is the high-level architecture?

Answer:

At a high level, the architecture is:

```
Balbix Web App
  |
API Gateway / Chat Middleware
  |
Auth + RequestContext Builder
  |
Chat Service
  |
AI Orchestrator
  |
  +--> Memory Service
  +--> Intent Router / Planner
  +--> Tool Execution Layer
  +--> RAG Service
  +--> LLM Gateway
  +--> Policy Engine
  +--> Response Validator
  +--> Audit / Observability
```

The important design principle is that the LLM is not directly connected to the database.

Instead, the orchestrator understands intent, the policy engine checks permissions, tools fetch live data, RAG fetches document evidence, the LLM generates the answer, and the validator checks safety and citations before returning the response.

8. What happens when a user sends a chat message?

Answer:

The request flow is:

1. User sends message from Balbix web app.
2. API middleware validates JWT.
3. System extracts user_id, tenant_id, and session_id.
4. Auth service fetches group IDs, role, and permissions.
5. RequestContext is created.
6. Chat service stores the user message.
7. AI orchestrator loads conversation context and memory.
8. Intent router decides whether tools, RAG, or both are needed.
9. Policy engine validates all planned actions.
10. Tools/RAG execute.
11. LLM generates final answer.
12. Response validator checks safety, citations, and RBAC.
13. Answer is streamed to UI.
14. Conversation history, audit logs, and memory extraction happen.

This flow ensures both product quality and enterprise security.

9. What is RequestContext and why is it important?

Answer:

RequestContext is a trusted identity and authorization object created after authentication.

It contains:

request_id
tenant_id
user_id
session_id
conversation_id
role
group_ids
permissions
allowed asset scopes
allowed document scopes
allowed actions
permission_version

This context is passed to every downstream component. It tells the system who is asking, which tenant they belong to, which groups they are part of, what data they can access, and what actions they are allowed to perform.

Without RequestContext, the AI layer may accidentally retrieve or expose unauthorized data.

10. Why did you use tools instead of only RAG?

Answer:

Most cybersecurity product data is structured and live.

Examples include:

```
Assets
Vulnerabilities
Risk scores
Exposure scores
Remediation status
Historical trends
Business units
Owners
```

This data should come from trusted backend APIs or SQL queries, not vector search.

For example, if the user asks:

```
Show top 10 risky assets.
```

The correct approach is to call a tool such as:

```
get_top_risky_assets()
```

That tool applies tenant filtering, group permissions, sorting, and business logic. RAG is useful for documents, but it is not the source of truth for live risk data.

So the design is:

```
Structured live data -> tools
Documents and PDFs -> RAG
Reasoning and explanation -> LLM
```

11. When do you use RAG?

Answer:

I use RAG when the answer depends on unstructured documents.

Examples include:

```
PDF reports
Security policies
Product documentation
Customer-uploaded documents
Remediation guidelines
Executive report templates
Vulnerability SLA policies
```

For example:

```
According to our policy, how quickly should critical vulnerabilities be fixed?
```

This should use RAG because the answer may be in a policy PDF.

But if the user asks:

```
Which critical vulnerabilities are currently open?
```

That should use tools because it requires live product data.

12. How do you handle a question that needs both tools and RAG?

Answer:

For mixed questions, I use both.

Example:

```
Using our SLA policy, show vulnerabilities currently violating SLA.
```

Flow:

```
1. RAG retrieves the SLA policy.
2. LLM or rule extractor converts policy into structured rules.
   Example: Critical = 7 days, High = 30 days.
3. Tool fetches current open vulnerabilities.
4. Backend compares vulnerability age against SLA.
5. LLM summarizes the result.
6. Answer includes policy citation and live data timestamp.
```

The important part is that deterministic comparison is done in backend code, not by sending thousands of vulnerability records to the LLM.

13. Why should the LLM not directly query the database?

Answer:

It is unsafe and hard to control. If the LLM directly queries the database, it may:

```
Generate unsafe SQL
Bypass tenant filters
Ignore RBAC
Fetch too much data
Leak sensitive information
Cause performance issues
```

Instead, we expose only approved backend tools such as:

```
get_risk_summary()
get_top_vulnerabilities()
get_asset_details()
get_risk_trend()
```

Each tool has schema validation, permission checks, tenant filters, rate limits, timeouts, audit logs, and output validation.

So the LLM can request a tool, but the backend decides whether it is allowed and how it executes safely.

14. How does the system prevent cross-tenant data leakage?

Answer:

Cross-tenant leakage is one of the biggest risks in this project.

We prevent it at multiple layers:

1. RequestContext always includes tenant_id.
2. Every DB query filters by tenant_id.
3. Every tool enforces tenant_id.
4. Every vector chunk has tenant metadata.
5. RAG retrieval filters by tenant_id and document scope.
6. Redis keys include tenant_id.
7. Audit logs include tenant_id.
8. Response validator checks that cited assets/docs belong to the same tenant.

Example Redis key:

```
conversation_context:{tenant_id}:{user_id}:{conversation_id}
```

The key idea is that the LLM should never receive data from another tenant in the first place.

15. How do you handle RBAC?

Answer:

RBAC is handled outside the LLM. After JWT validation, we fetch the user role, group IDs, and permissions, then build RequestContext.

Example:

```
user_id = U1
tenant_id = T1
group_ids = [G1, G2]
allowed_asset_groups = [A1, A2]
permissions = can_read_assets, can_read_risk
```

Every tool uses this context. For example, an asset query includes:

```
WHERE tenant_id = :tenant_id
AND asset_group_id IN (:allowed_asset_groups)
```

RAG uses the same context to retrieve only allowed documents. So even if the user asks to see all assets, the tool only returns assets that the user is allowed to see.

16. How do you handle memory in this chatbot?

Answer:

There are two types of memory.

First is session/conversation memory. This helps with follow-up questions like:

```
Now summarize this for my VP.
```

Second is long-term user memory. This stores safe, stable preferences such as:

```
User prefers executive summaries.
```

```
User prefers PDF reports.
User prefers business-impact explanations.
```

We do not store sensitive cyber facts as long-term memory.

Bad memory example:

```
Tenant X has CVE-123 on production database.
```

That type of information must always be fetched live from tools. Memory is used only for personalization, not as source of truth.

17. When do you decide to store long-term memory?

Answer:

Memory extraction happens after the assistant response is completed.

Flow:

```
User asks question
System answers normally
After response, async memory extraction runs
It decides whether something should be stored
```

For explicit memory, such as:

```
Remember that I prefer CISO-style reports.
```

we can store directly.

For inferred memory, we use a lightweight LLM or classifier. If the user repeatedly asks to make responses business-friendly, summarize for leadership, or keep things non-technical, the memory extractor may store:

```
User prefers business-facing risk explanations.
```

But we store only if the memory is safe, stable, and useful.

18. How do you handle multiple user requests in one message?

Answer:

I use a Task Decomposer and Workflow Planner.

Example:

```
Analyze this PDF, compare it with current vulnerabilities, generate a report, and create Jira tickets.
```

This is decomposed into tasks:

```
Task 1: Analyze PDF
Task 2: Compare PDF findings with current vulnerabilities
Task 3: Generate report
Task 4: Prepare Jira ticket drafts
Task 5: Ask for approval
Task 6: Create Jira tickets after approval
```

Some tasks are sequential, and some can run in parallel. The planner creates a task graph/DAG, and the policy engine checks each step before execution.

This is better than giving the entire instruction directly to the LLM.

19. How do you handle repeated LLM-tool loops?

Answer:

I use a bounded agent loop.

The flow is:

```
LLM decides it needs a tool
Orchestrator validates tool request
Policy engine checks permission
Tool executes
Tool result goes back to LLM
LLM decides next step
```

This can repeat, but only within strict limits.

Example limits:

```
max_tool_iterations = 5
max_total_latency = 40 seconds
max_parallel_tools = 3
```

The loop stops when:

```
Final answer is ready
Clarification is needed
Approval is needed
Tool fails
Max iterations reached
Request becomes async
```

This gives agentic behavior while preventing infinite loops and unsafe execution.

20. What are the most important design principles in this project?

Answer:

The most important principles are:

```
1. Tools are source of truth for live structured data.
2. RAG is source of evidence for documents.
3. LLM is used for reasoning and explanation, not direct data access.
4. RBAC and tenant isolation are enforced outside the LLM.
5. Every factual answer should be grounded with evidence.
6. Memory stores preferences, not sensitive cyber facts.
7. Write actions like Jira creation require approval.
8. The system should say "I don't know" when evidence is missing.
9. Redis is cache; durable DB is source of truth.
10. Every request must be observable, auditable, and evaluatable.
```

These principles make the system enterprise-grade and production-safe.

Batch 1 Summary

This batch prepares you to explain the project from first principles: why it exists, how the architecture is shaped, and why security boundaries matter more than simply connecting a UI to an LLM.

In an interview, the most important signal is that you can clearly separate responsibilities: tools fetch trusted live facts, RAG retrieves document evidence, memory personalizes safely, policy controls access, and the LLM explains the result.